

Good evening everyone



class start at 8:08

Q) Print 1 to 100 number using separate thread.

sleep(): this method used to pauses the execution of the currently running thread.

sleep(long millisecond)

1000 → 1 sec

* this method can throw an interrupted exception

- 1) try and catch
- 2) throws exception

```
main() {
```

```
    int sec = 10;
```

```
    while( sec > 0 ) {
```

```
        syso(sec);
```

```
        take pause for one second
```

```
        sec--;
```

```
    }
```

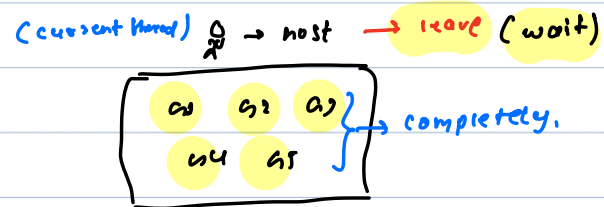
```
    syso ("time's up");
```

```
}
```

10 } → **sec**
?
:
1

main thread is pause for some time.

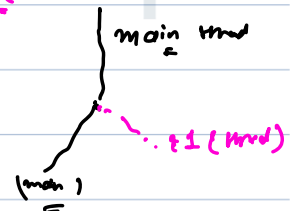
Join() : this method is wait for another method to finish its execution.



main () {

```
my task task = new mytask1();  
Thread t1 = new Thread ( task );  
t1.start();  
t1.join(); // wait till completely executing  
for ( int i=0; i<10; i++)  
    print ( );
```

}



3

main thread is waiting till t1 is not completely executed.
then it continue.

Q) Print 1 to 10000 number using separate thread.

we have to create 10,000 threads.

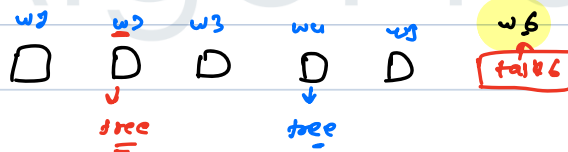
organize a wedding. (user)

task (decoration, catering, music...)

• user is responsible to manage all the task.

• At task increases, you hire a new worker (new thread) to handle each task.

• This can lead to inefficient resource utilization.



to deal all the task.

executor

(event manager)

it take all the responsibility

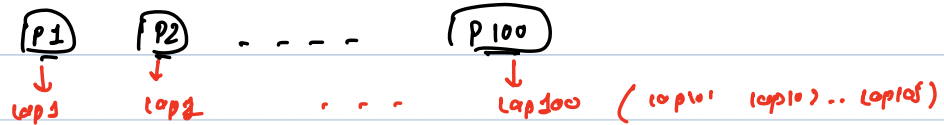
it provide a high level abstraction for managing task.

it improve your system performance.



How exactly executor manage all the threads.

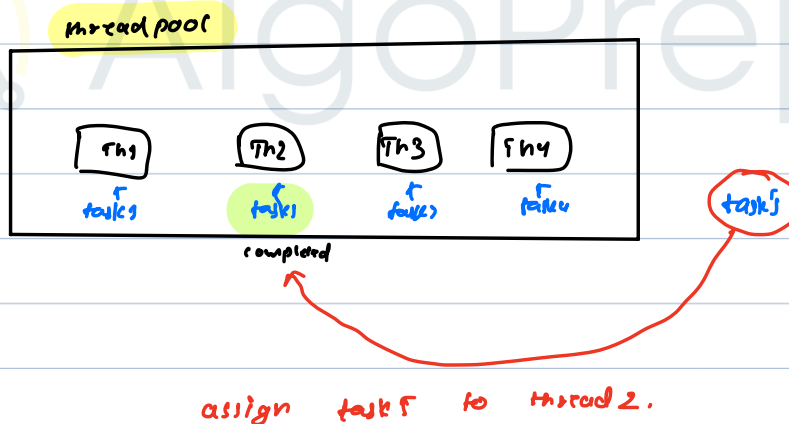
you have to manufacture 100 laptops. (task)



(100,000) → 100,000 production unit

thread pool
(pool of threads)

its reuse the available threads.



instead of creating or destroying threads for each task, a threadpool reuse the existing thread.

break Hl 9:14

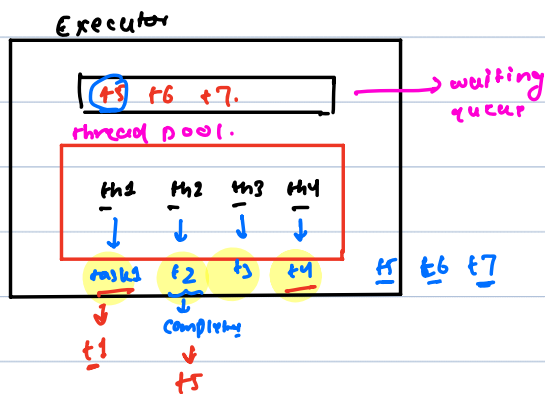
threadpool

fixed

dynamic

In this we have fixed number of threads.

thread are pre-created

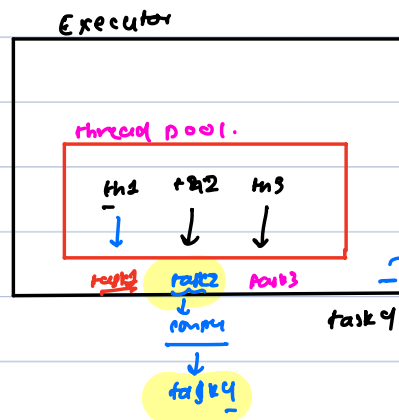


where task is known.

it is not efficient if some of threads is always idle.

`new FixedThreadPool()`

In this it is depend on the demand of the task.



threads created on demand and if there is no available threads.

if any thread idle then it automatic destroy.

`new CachedThreadPool();`

```
main ( ) {
```

```
ExecutorService manager = Executor = new FixedThreadPool(5)
```

```
for ( int i=1; i<=1000; i++) {
```

```
    myThread task = new myThread(i);
```

```
    manager.submit( task);
```

```
}
```

```
manager.shutdown();
```

```
}
```

Runnable → its not return any result.

if you want to return something.



callable

```
class myCallable implements Callable<Integer> {
```

```
    private int num;
```

```
    myCallable (int n) {
```

```
        this.num = n;
```

```
    }
```

```
    public Integer call() throws Exception {
```

```
        int s = 0;
```

```
        for (int i = 1; i <= num; i++) {
```

```
            s += i;
```

```
        }
```

```
        return s;
```

```
    }
```

```
}
```

```
main ( ) {
```

```
    ExecutorService manager = Executors.newCached  
                                ThreadPool();
```

```
    Callable<Integer> task = new myCallable(10);
```

```
    myCallable task = new myCallable(10);
```

```
    Future<Integer> fut = manager.submit(task);
```

```
    syso ( fut.get());
```

```
}
```



future

executor return future object.

```
public class task {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        //count down
        int sec = 10;
        System.out.println(x:"Countdown start");
        while(sec>0){
            System.out.println("Remaining sec "+sec);
            Thread.sleep(millis:1000);
            sec--;
        }
        System.out.println(x:"Time's up");
    }
}

1 import java.util.concurrent.ExecutorService;
2 import java.util.concurrent.Executors;
3
4 class thread5 implements Runnable{
5
6     //create constructor
7     private int number;    //1 , 2, 3 ,..100
8     public thread5(int num){
9         this.number = num;
10    }
11    public void run(){
12        System.out.println("my task is :" + number + Thread.currentThread().getName());
13    }
14 }
15
16 public class task1 {
17     public static void main(String[] args) throws InterruptedException {
18         // main thread
19         System.out.println("start...");
20         // ExecutorService manager = Executors.newFixedThreadPool(10);
21         ExecutorService manager = Executors.newCachedThreadPool();
22
23         for(int i = 1; i<=1000; i++){
24             thread5 tk = new thread5(i); //create the task
25             manager.submit(tk);
26             // Thread t5 = new Thread(tk);
27             // t5.start();
28             // Thread.sleep(100); // main thread is pause
29         }
30         manager.shutdown();
31         System.out.println("this is end");
32     }
33 }
34 }
```

```
1  import java.util.concurrent.Callable;
2  import java.util.concurrent.ExecutorService;
3  import java.util.concurrent.Executors;
4  import java.util.concurrent.Future;
5
6  class Mycallable implements Callable<Integer>{
7      private int num;
8      public Mycallable(int n){
9          this.num = n;
10     }
11
12     public Integer call(){
13         int s = 0;
14         for(int i = 1; i<=num; i++){
15             s+=i;
16         }
17         return s;
18     }
19 }
20 public class task2 {
21     Run | Debug
22     public static void main(String[] args) throws Exception{
23         int n = 10;
24         Mycallable task = new Mycallable(n); // task created
25         ExecutorService manager = Executors.newCachedThreadPool();
26         Future<Integer> future = manager.submit(task);
27         System.out.println("sum of "+n+ " is :" + future.get());
28         manager.shutdown();
29     }
30 }
31
```